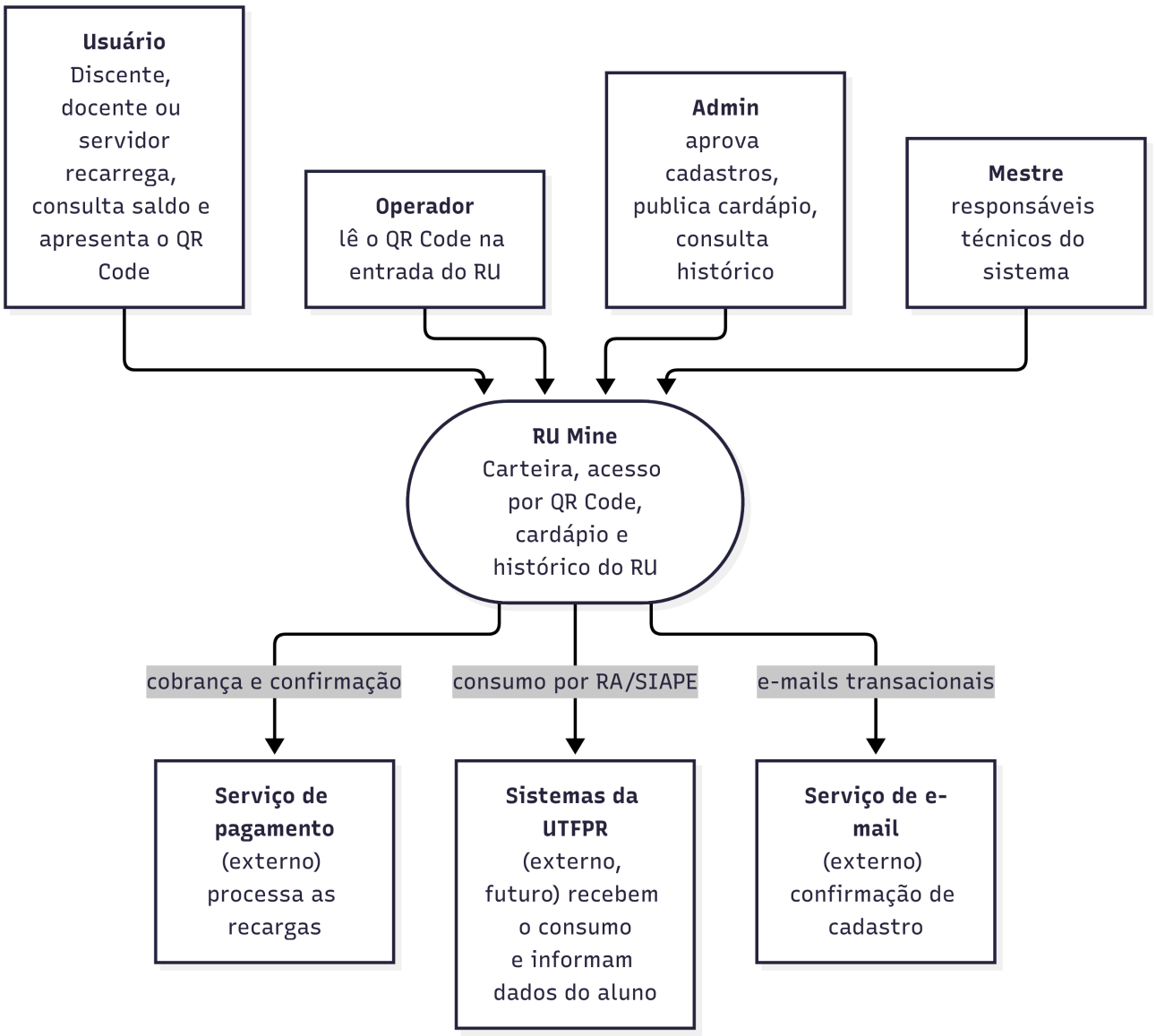
RU Mine: Semana 3 - arquitetura, ADRs e modelo de dados

1. Decisões da equipe

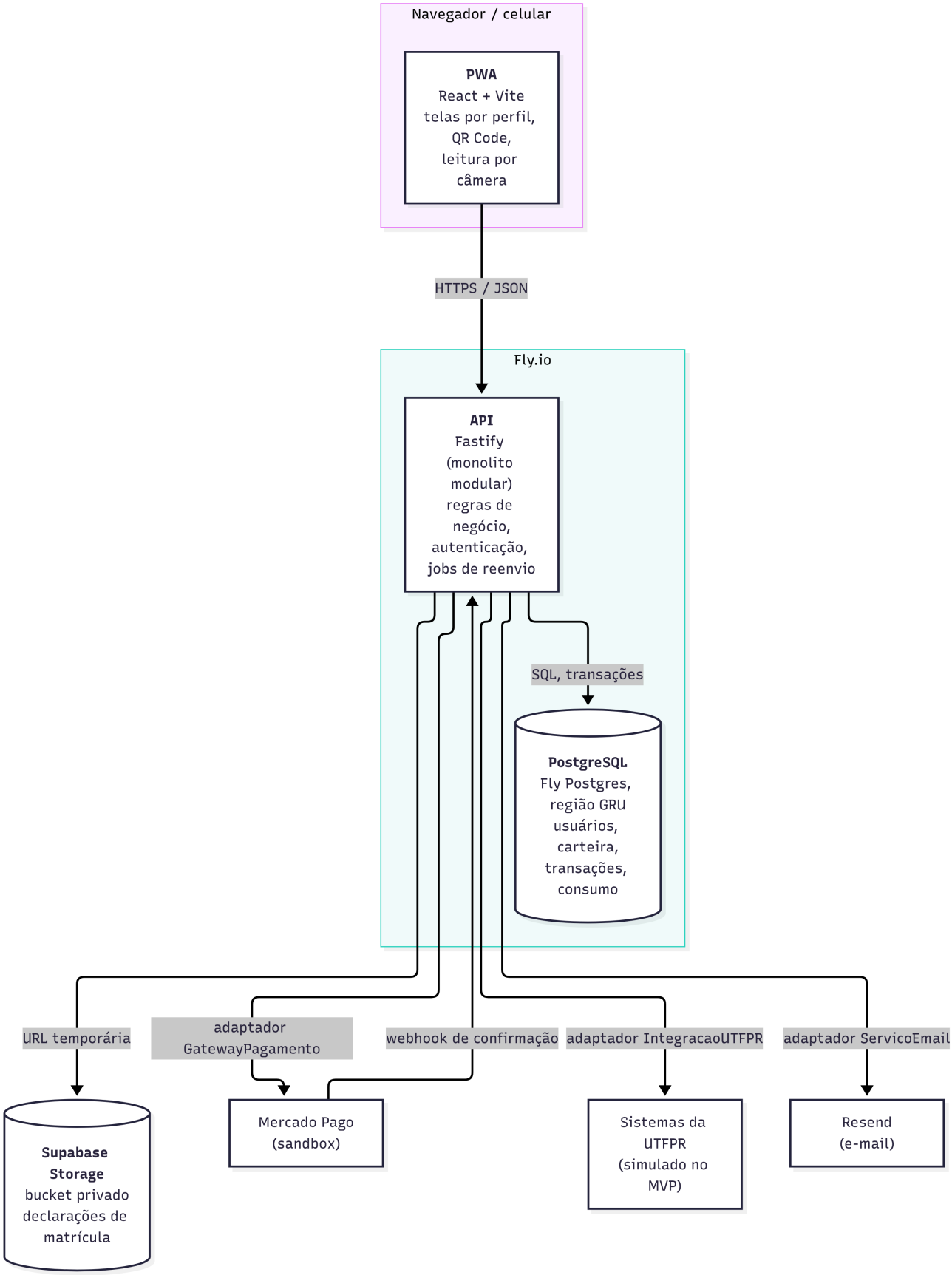
Tema	Decisão
Pilha	A: TypeScript de ponta a ponta
Backend	**Fastify** com arquitetura modular manual (sem NestJS)
Acesso a dados	**Prisma**
Cliente	PWA (React + Vite)
Banco	PostgreSQL (**Fly Postgres**, região **GRU**, São Paulo)
Deploy	Fly.io, em contêiner Docker
Arquivos	**Supabase Storage** (bucket privado)
E-mail	**Resend**
Pagamento	**Sandbox do Mercado Pago**, com confirmação por webhook
Integração UTFPR	Adaptador **simulado** no MVP; respostas reais ficam para depois do MVP
QR Code	Token JWT de **30 segundos**
Senha	Mínimo de 8 caracteres, com número e caractere especial; hash com **Argon2**
Arquivo de declaração	Máximo de **2 MB**; apenas `.pdf`, `.jpg` e `.png`
Estilo	Monolito modular
Diagramas	C4 Model

2. Diagramas C4

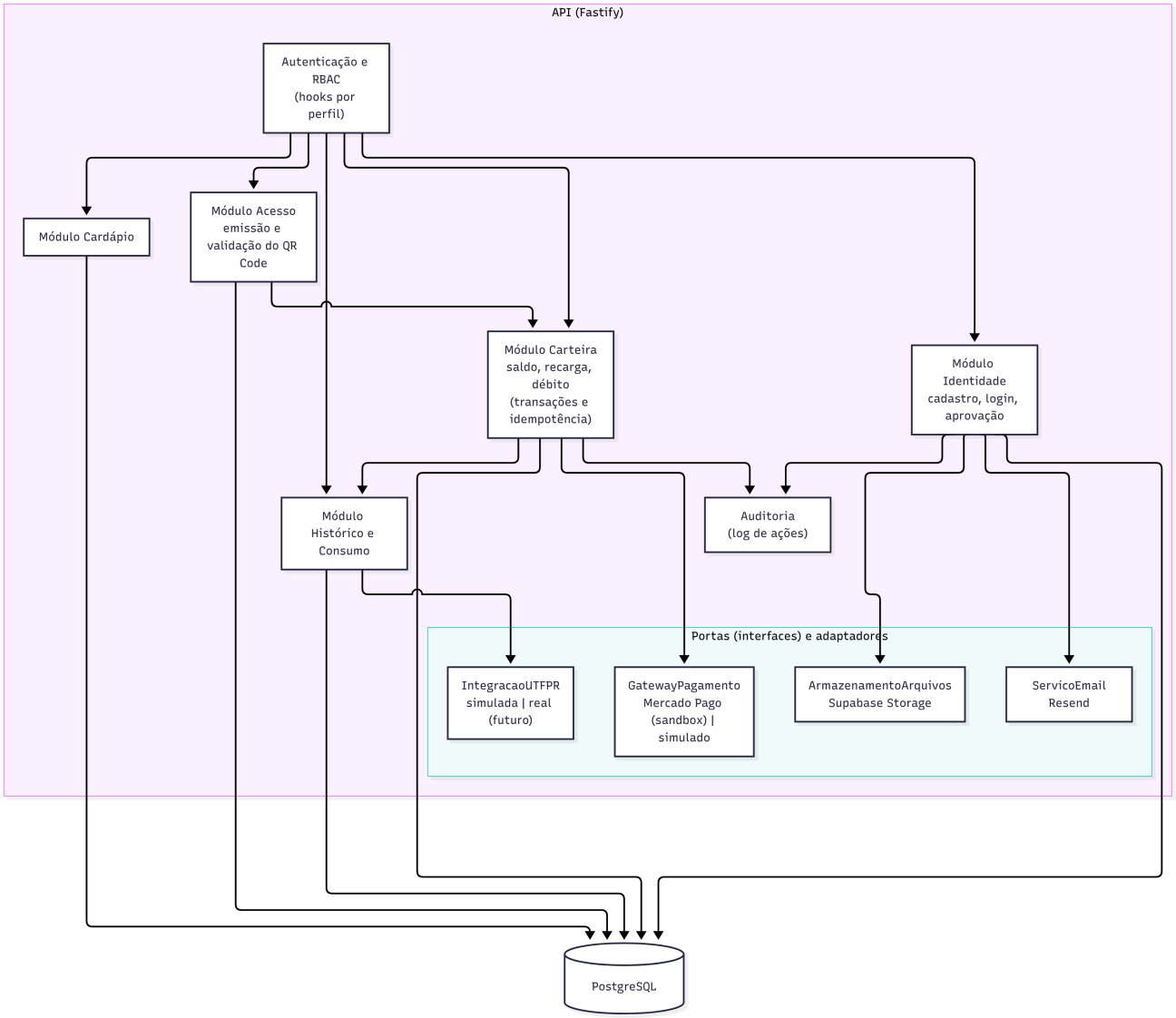
2.1 Nível 1: contexto



2.2 Nível 2: contêineres

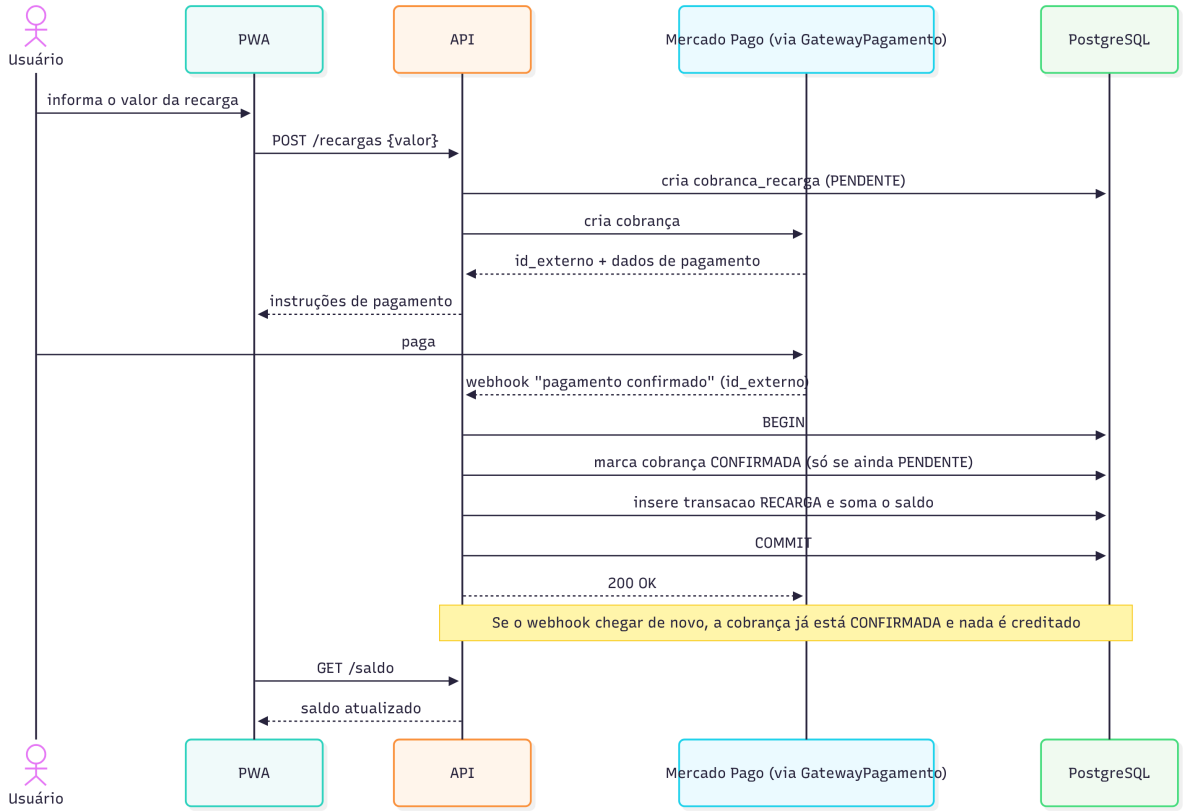


2.3 Nível 3: componentes da API

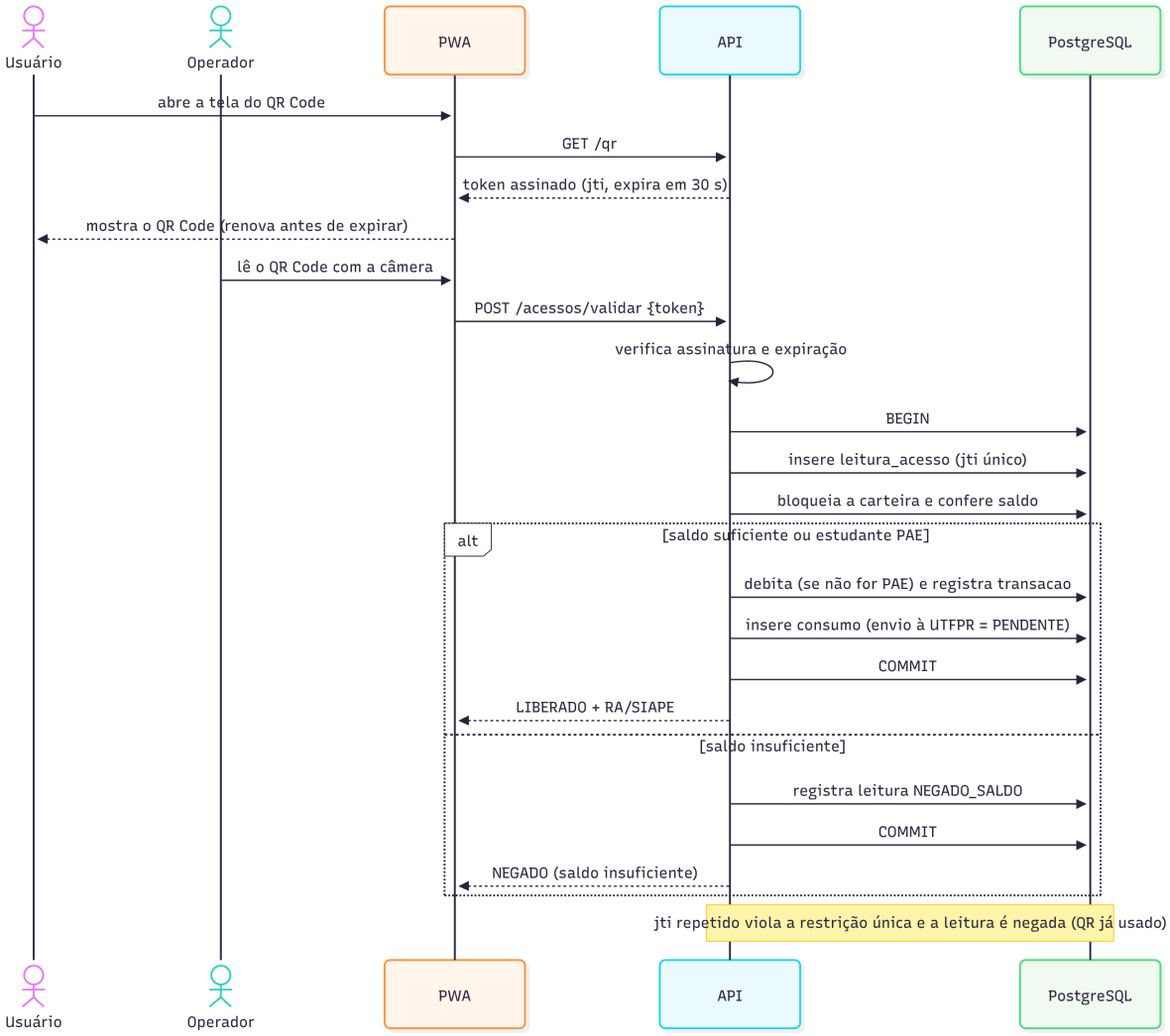


3. Diagramas de sequência dos fluxos de maior risco

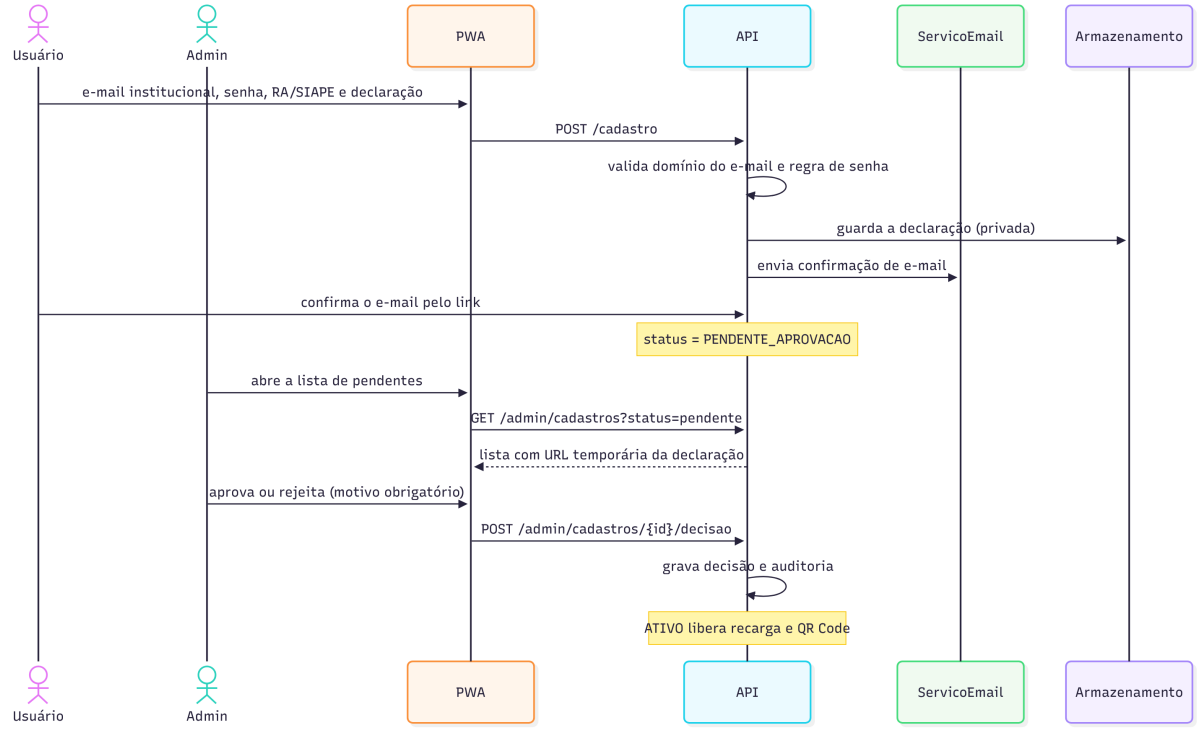
3.1 Recarga com confirmação idempotente



3.2 Leitura do QR Code e débito

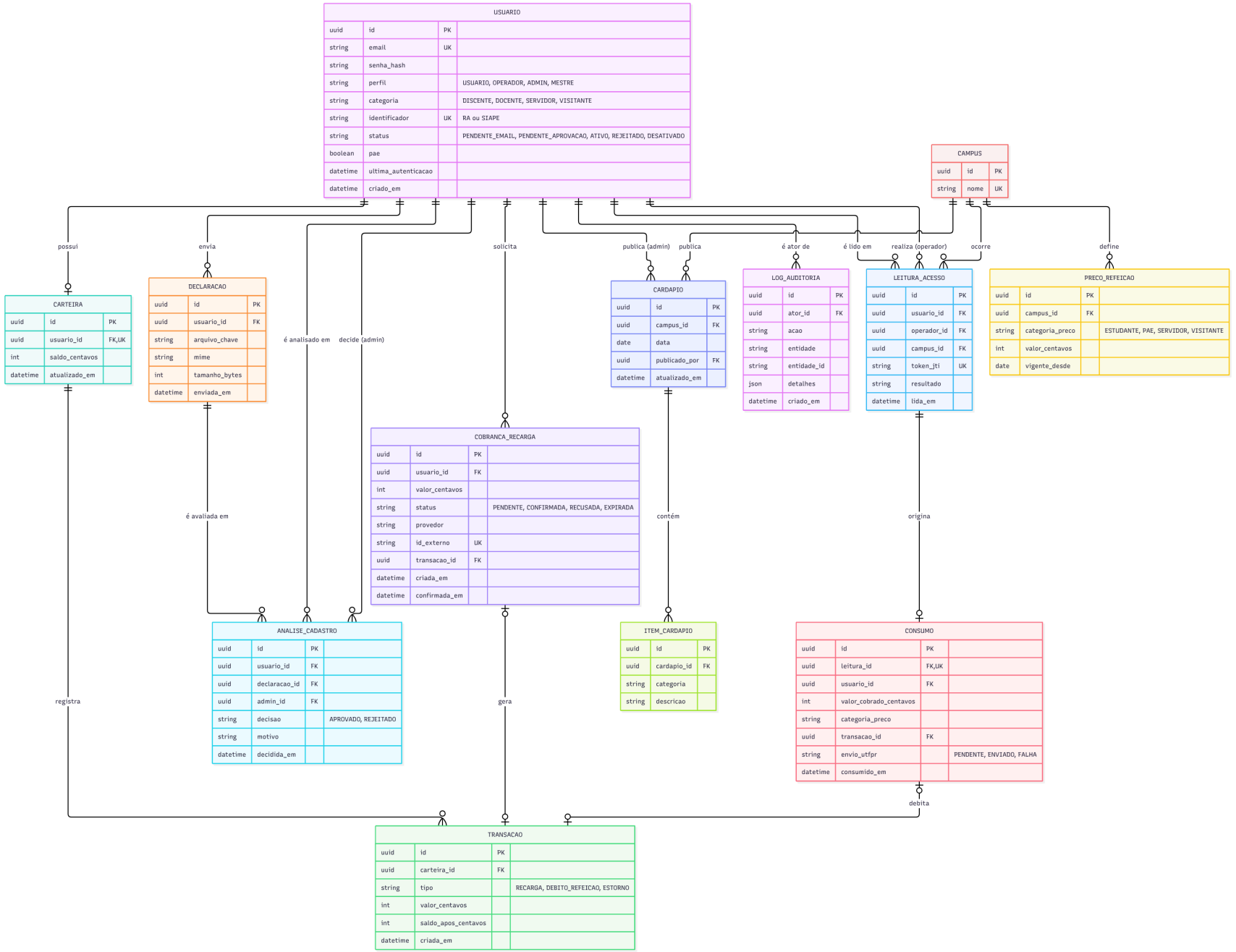


3.3 Cadastro e aprovação



4. Modelo entidade-relacionamento

Valores monetários em **centavos inteiros** (evita erro de arredondamento). Saldo e histórico formam um **livro-razão**: `transacao` guarda cada movimento e o saldo resultante, e `carteira.saldo_centavos` é o valor corrente. O arquivo `rumine.dbml` traz o mesmo modelo para colar em dbdiagram.io.


USUARIO
uuid id PK
string email UK
string senha_hash
string perfil USUARIO, OPERADOR, ADMIN, MESTRE
string categoria DISCENTE, DOCENTE, SERVIDOR, VISITANTE
string identificador UK RA ou SIAPE
string status PENDENTE_EMAIL, PENDENTE_APROVACAO, ATIVO, REJEITADO, DESATIVADO
boolean pae
datetime ultima_autenticacao
datetime criado_em
CAMPUS
uuid id PK
string nome UK
possui
envia
é analisado em
decide (admin)
solicita
publica (admin)
publica
é ator de
é lido em
realiza (operador)
ocorre
define
CARTEIRA
uuid id PK
uuid usuario_id FK,UK
int saldo_centavos
datetime atualizado_em
DECLARACAO
uuid id PK
uuid usuario_id FK
string arquivo_chave
string mime
int tamanho_bytes
datetime enviada_em
CARDAPIO
uuid id PK
uuid campus_id FK
date data
uuid publicado_por FK
datetime atualizado_em
LOG_AUDITORIA
uuid id PK
uuid ator_id FK
string acao
string entidade
string entidade_id
json detalhes
datetime criado_em
LEITURA_ACESSO
uuid id PK
uuid usuario_id FK
uuid operador_id FK
uuid campus_id FK
string token_jti UK
string resultado
datetime lida_em
PRECO_REFEICAO
uuid id PK
uuid campus_id FK
string categoria_preco ESTUDANTE, PAE, SERVIDOR, VISITANTE
int valor_centavos
date vigente_desde
é avaliada em
COBRANCA_RECARGA
uuid id PK
uuid usuario_id FK
int valor_centavos
string status PENDENTE, CONFIRMADA, RECUSADA, EXPIRADA
string provedor
string id_externo UK
uuid transacao_id FK
datetime criada_em
datetime confirmada_em
contém
origina
ANALISE_CADASTRO
uuid id PK
uuid usuario_id FK
uuid declaracao_id FK
uuid admin_id FK
string decisao APROVADO, REJEITADO
string motivo
datetime decidida_em
ITEM_CARDAPIO
uuid id PK
uuid cardapio_id FK
string categoria
string descricao
CONSUMO
uuid id PK
uuid leitura_id FK,UK
uuid usuario_id FK
int valor_cobrado_centavos
string categoria_preco
uuid transacao_id FK
string envio_utfpr PENDENTE, ENVIADO, FALHA
datetime consumido_em
registra
gera
debita
TRANSACAO
uuid id PK
uuid carteira_id FK
string tipo RECARGA, DEBITO_REFEICAO, ESTORNO
int valor_centavos
int saldo_apos_centavos
datetime criada_em

Regras de integridade que o banco deve garantir:

- `carteira.saldo_centavos >= 0` (restrição `CHECK`); o débito usa bloqueio da linha (`SELECT ... FOR UPDATE`) na mesma transação.
- `leitura_acesso.token_jti` único: impede o uso duas vezes do mesmo QR Code (RNF-12).
- `cobranca_recarga.id_externo` único e transição `PENDENTE → CONFIRMADA` condicional: impede crédito duplicado (RNF-03).
- `consumo.leitura_id` único: um consumo por leitura liberada (RNF-13: reenvio sem duplicar).
- `cardapio` único por `(campus_id, data)`.
- `usuario.identificador` único (RA ou SIAPE); `usuario.email` único.

5. ADRs (registros de decisão de arquitetura)

Todos os ADRs abaixo refletem decisões da equipe.

ADR-001: Estilo de arquitetura (monolito modular)

- **Status:** Aceito
- **Contexto:** Equipe de 5 pessoas, MVP em 2 semanas, saldo e débito exigem consistência forte e a validação do QR Code deve ser rápida.
- **Opções:** (1) monolito modular; (2) microsserviços; (3) funções serverless.
- **Decisão:** Monolito modular, com módulos Identidade, Carteira, Acesso, Cardápio e Histórico/Consumo, e as integrações externas atrás de interfaces.
- **Consequências:** Transações simples e deploy único. Os módulos devem depender uns dos outros só por interfaces, para permitir separar serviços no futuro. A API escala como um bloco só.

ADR-002: Backend com Fastify e módulos manuais

- **Status:** Aceito (substitui a proposta de NestJS)
- **Contexto:** A equipe adotou TypeScript. O NestJS exige decoradores, injeção de dependência avançada e RxJS, o que pesa em um MVP de 2 semanas.
- **Opções:** NestJS; **Fastify**; Express.
- **Decisão: Fastify** com estrutura modular manual: uma pasta por módulo (`identidade`, `carteira`, `acesso`, `cardapio`, `historico`), cada uma como plugin do Fastify com rotas, serviço e repositório. Dependências injetadas por parâmetro (funções de fábrica), sem contêiner de injeção. Validação de entrada por esquema (por exemplo JSON Schema ou Zod) e autorização por perfil em hooks.
- **Consequências:** Curva de aprendizado menor e menos “mágica”. Em compensação, a disciplina de módulos fica por conta da equipe: sem regra automática, é fácil um módulo acessar o banco de outro. Combinem uma regra simples (módulos só chamam outros pelo serviço público) e revisem nos pull requests.

ADR-003: Frontend como PWA

- **Status:** Aceito
- **Contexto:** Usuários e operador usam o celular; o operador precisa da câmera; um aplicativo nativo não cabe no prazo.

- **Opções:** PWA com React; aplicativo nativo (React Native, Flutter); site responsivo sem instalação.
- **Decisão: PWA** com React e Vite, com manifesto e service worker (por exemplo `vite-plugin-pwa`). QR Code gerado no navegador por biblioteca de QR; leitura pela API de câmera do navegador ou biblioteca de leitura (a confirmar em teste).
- **Consequências:** Um só código para todos os perfis e instalação pela tela inicial. A câmera exige HTTPS. O suporte de PWA varia entre navegadores: testar no iOS e no Android antes de fechar a biblioteca de leitura.

ADR-004: PostgreSQL com Prisma

- **Status:** Aceito
- **Contexto:** Saldo, débito e recarga exigem atomicidade e restrições de integridade; a equipe valoriza tipagem automática e migrações simples.
- **Opções:** PostgreSQL com Prisma; PostgreSQL com TypeORM; MySQL; MongoDB.
- **Decisão: PostgreSQL**, valores em centavos inteiros, restrições `CHECK` e `UNIQUE` e bloqueio de linha no débito. Acesso por **Prisma**, com migrações versionadas.
- **Consequências:** Tipos gerados do esquema e migrações fáceis. O débito precisa de transação interativa com bloqueio de linha (`SELECT ... FOR UPDATE`), que o Prisma não expressa na API de alto nível: usar `$queryRaw` dentro de `$transaction`. Escrever testes de concorrência para esse trecho.

ADR-005: Autenticação, sessão e QR Code

- **Status:** Aceito
- **Contexto:** Senha própria, reautenticação a cada 45 dias e QR Code de curta duração sem expor o RA.
- **Opções de sessão:** cookie de sessão `HttpOnly`; JWT no armazenamento do navegador.
- **Decisão:**
  - **Senha:** mínimo de 8 caracteres, com pelo menos um número e um caractere especial; hash com **Argon2**.
  - **Sessão:** cookie `HttpOnly`, `Secure` e `SameSite`, com validade máxima de 45 dias.
  - **QR Code:** token JWT assinado com `sub`, `jti` e expiração de **30 segundos**; nunca contém o RA. O uso único vem da restrição única do `jti` no banco.
- **Consequências:** Boa proteção contra roubo de token por script. É preciso proteger contra CSRF e manter o relógio do servidor correto. Como o token vive 30 segundos, a PWA renova o QR Code pouco antes de expirar (por exemplo a cada 25 s) e o servidor aceita uma pequena tolerância de relógio. O sistema nunca coleta a senha institucional da UTFPR (RNF-14).

ADR-006: Pagamento com sandbox do Mercado Pago

- **Status:** Aceito
- **Contexto:** A equipe quer a recarga por Pix e uma demonstração realista; o provedor tem documentação em português.
- **Opções:** só simulado; sandbox do Mercado Pago; outro provedor.
- **Decisão:** Interface `GatewayPagamento` com implementação do **sandbox do Mercado Pago**, confirmando o pagamento por **webhook** tratado de forma idempotente. A implementação **simulada** continua disponível no mesmo contrato, para testes automáticos e como plano B.

- **Consequências:** Demonstração mais realista e sem risco financeiro. Riscos: (1) o webhook precisa de URL pública (no Fly.io existe; em desenvolvimento local exige um túnel); (2) o sandbox deve ser validado logo no início do desenvolvimento, com o fluxo de Pix e o disparo do webhook de confirmação; se não funcionar como esperado, o adaptador simulado é usado na apresentação.

ADR-007: Integração com a UTFPR (adaptador simulado)

- **Status:** Aceito
- **Contexto:** Obter credenciais e documentação de API da UTFPR em 2 semanas bloquearia a equipe.
- **Opções:** esperar a UTFPR; adaptador simulado com exportação; scraping (não recomendado).
- **Decisão:** Interface `IntegracaoUTFPR` com implementação **simulada**. O `consumo` guarda o estado de envio (`PENDENTE/ENVIADO/FALHA`) e há exportação de consumos por período. As perguntas à UTFPR ficam para o ciclo pós-MVP.
- **Consequências:** A lógica de reenvio sem duplicar (RNF-13) é validada já no MVP. A integração real pode mudar o formato do envio. Nunca coletar credenciais da UTFPR.

ADR-008: Arquivos com Supabase Storage e e-mail com Resend

- **Status:** Aceito
- **Contexto:** Declarações de matrícula são dados pessoais (LGPD) e a confirmação de cadastro é por e-mail.
- **Opções de arquivos:** Supabase Storage; S3 da AWS; disco da aplicação.
- **Decisão:**
  - **Arquivos: Supabase Storage** em bucket **privado**, com URL temporária de leitura só para o Admin. Apenas `.pdf`, `.jpg` e `.png`, máximo de **2 MB**, validados no servidor (tipo real e tamanho, não só a extensão).
  - **E-mail: Resend** atrás de `ServicoEmail`; em desenvolvimento, o link aparece no log.
- **Consequências:** Configuração simples e cota gratuita adequada ao MVP. O Supabase é usado só para arquivos, não para banco nem autenticação, para não duplicar responsabilidades. É preciso definir o prazo de retenção das declarações (RNF-15). Em produção, o Resend exige verificar o domínio de envio.

ADR-009: Deploy no Fly.io com Fly Postgres e CI

- **Status:** Aceito
- **Contexto:** A equipe escolheu Fly.io; a validação do QR Code deve ser rápida e o banco precisa ficar perto da API.
- **Opções:** Fly.io; Heroku; AWS.
- **Decisão:** Aplicação em contêiner **Docker** no **Fly.io**, com **Fly Postgres** (cluster não gerenciado) na região **GRU** (São Paulo), junto da API. **GitHub Actions** executa testes e verificação de estilo a cada pull request. O deploy é configurado desde o início do desenvolvimento.
- **Consequências:** Baixa latência entre API e banco, ajudando o RNF-04. Como o Fly Postgres não é gerenciado, a equipe cuida de **backups e restauração** e deve testar uma restauração antes da apresentação.

6. Pilha tecnológica consolidada

Camada	Escolha	Status
Linguagem	TypeScript	Aceito
Backend	Fastify, módulos manuais	Aceito
Frontend	React + Vite, PWA	Aceito
Banco	PostgreSQL (Fly Postgres, GRU) + Prisma	Aceito
Autenticação	Argon2 + cookie de sessão; QR Code com JWT de 30 s	Aceito
Arquivos	Supabase Storage (privado)	Aceito
E-mail	Resend	Aceito
Pagamento	Mercado Pago (sandbox) via webhook, com adaptador simulado	Aceito
Integração UTFPR	Adaptador simulado	Aceito
Testes	Vitest ou Jest (unitários e API); Playwright (ponta a ponta)	A decidir
Contêiner e deploy	Docker + Fly.io	Aceito
CI	GitHub Actions	Aceito
Gestão	GitHub Projects	Aceito

7. Pontos conferidos e plano B

A equipe confirmou os pontos abaixo antes de iniciar o desenvolvimento.

Ponto	Situação	Plano B
Sandbox do Mercado Pago com Pix e webhook de confirmação	Confirmado	Adaptador de pagamento simulado
Resend: limite do plano gratuito e domínio de envio	Confirmado	Registrar o link de confirmação no log em desenvolvimento
Supabase Storage: cota gratuita e bucket privado com URL temporária	Confirmado	Armazenamento compatível com S3 por outro provedor
Fly Postgres (cluster não gerenciado) na região GRU	Confirmado	Banco gerenciado de outro provedor, mantendo o PostgreSQL